

Arm[®] GPU Bug Bounty How-To Guide

Public

Copyright © 2025 Arm Limited (or its affiliates). All rights reserved.

Document Version: 20250623-1.0

- [System Diagram](#)
- [Using the Arm[®] Mali[™] SW Drivers](#)
 - [Example usage of Kbase](#)
 - [What is the User Shared Memory used for?](#)
 - [What is the Kbase Shared Memory used for?](#)
 - [How do you access the Mali GPU Kernel Driver 'Kbase' through the U/K interface?](#)
 - [What are the Mali GPU Memory Management syscalls?](#)
 - [What are the GPU Queue Management Syscalls?](#)
 - [What are the Synchronization Operations and Syscalls?](#)
 - [What are KCPU Queue Management Syscalls?](#)
 - [Details of Miscellaneous Mali syscalls](#)
- [Using the Mali GPU Firmware](#)
 - [How do you access the Command Stream Frontend \(CSF\) and GPU Firmware, through Command Buffers?](#)
 - [Decoding the Firmware Image](#)
- [Proprietary notice](#)

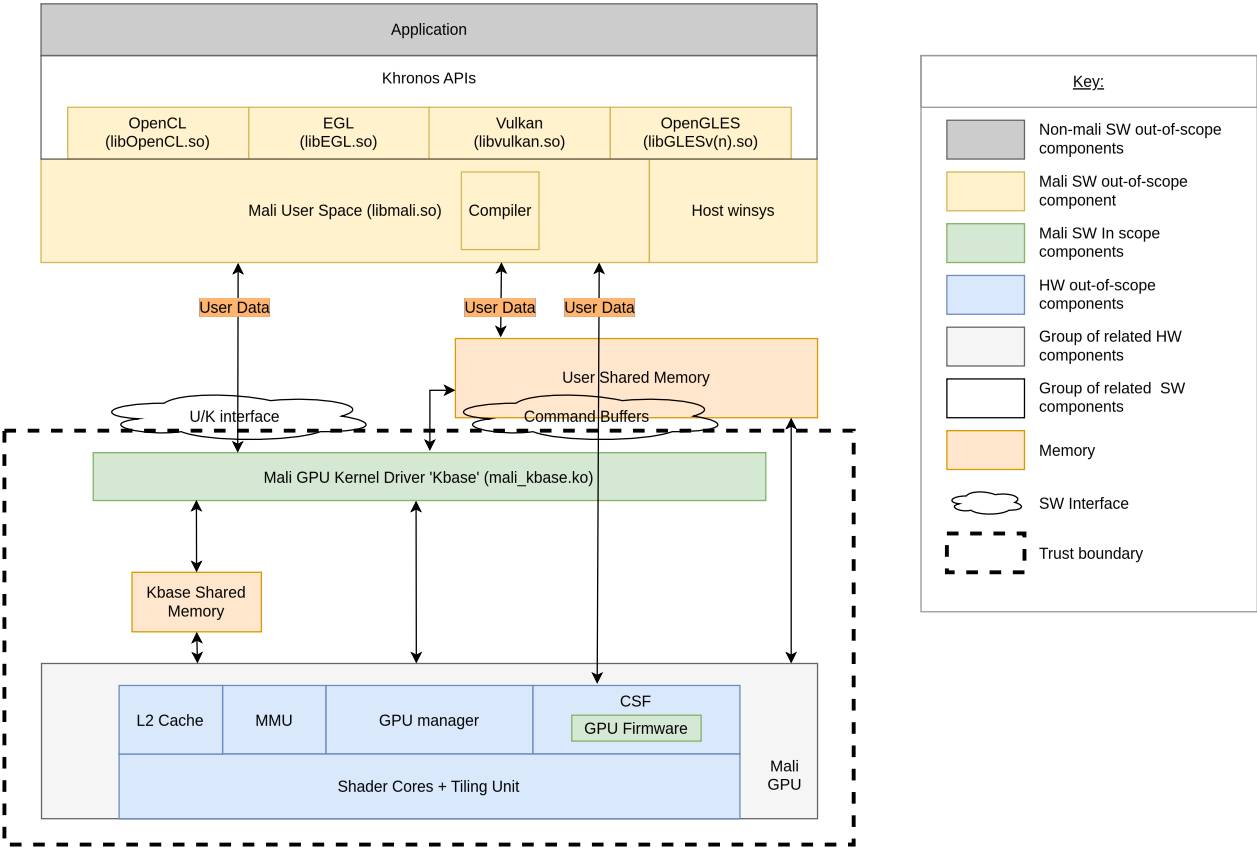
This document is Non-Confidential.

Copyright © 2025 Arm Limited or its affiliates. All rights reserved.

This document is protected by copyright and other intellectual property rights. Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary notice](#) found at the end of this document.

These materials and information are provided as part of Arm's Bug Bounty program to assist security researchers in their vulnerability research related to this program. These documents do not constitute official Arm documentation, and no warranties or guarantees are provided regarding their accuracy, completeness, or suitability for any purpose outside of the Bug Bounty context. Arm reserves the right to modify or withdraw any materials at its discretion without prior notice.

System Diagram



Using the Arm[®] Mali[™] SW Drivers

The following variable will be used to refer to the directory containing your Linux Kernel source:

```
KDIR
```

The location of the Mali GPU Kernel Driver 'Kbase' is:

```
$KDIR/drivers/gpu/arm/midgard
```

Example usage of Kbase

The following contain example usage and/or explanations of Kbase in terms of GPU Memory allocation and mapping, and Command Stream Group/Queue allocation. They exploit previous vulnerabilities in the Mali Kernel Driver that have since been patched.

These can be used to understand the basic operation of the driver and to make some initial syscalls to allocate and manipulate GPU memory:

- [GitHub Security Lab PoCs for Mali](#) - for a small selection of CVEs
- [Black Hat Asia 2023: Two Bugs With One PoC: Rooting Pixel 6 From Android 12 to Android 13](#) - Yong Wang (Security Engineer, Alibaba Security Pandora Lab)
- [Black Hat USA 2023: Make KSMA Great Again: The Art of Rooting Android Devices by GPU MMU Features](#) - Yong Wang (Security Engineer, Alibaba Cloud Pandora Lab)
- [Project Zero 42451457: Arm Mali CSF: page freed while still mapped into host userspace due to VMA split mishandling](#)

PoC from Project Zero: Downloading and compiling

Refer to <https://project-zero.issues.chromium.org/issues/42451457> and download the [mali_test.c](#) attachment.

Due to an update in recent drivers, the value in the initialization of `struct kbase_ioctl_cs_queue_register_cqr`, circa line 190, must be changed to:

```
.buffer_size = 0x1000,
```

Compile as follows:

```
gcc -march=native mali_test.c -o mali_test
```

Enable some dynamic debug for the lines:

```
sudo su
alias ddcmd='echo $* > /sys/kernel/debug/dynamic_debug/control'
# Look for the 'unexpected' messages on user_io pages:
cat /sys/kernel/debug/dynamic_debug/control | grep 'mali_kbase_mem_linux.c' |
grep -i unexpected | grep -i pages

# e.g. for r54p0 5th Gen kbase, this is lines 3328-3343
ddcmd 'file drivers/gpu/arm/midgard/mali_kbase_mem_linux.c line 3328-3343
+pt'
```

Run the resulting executable. on dmesg you should see the following message repeated:

```
mali mali.0: Attempt to free GPU memory whose freeing by user space is
forbidden!
```

...finally ending in:

```
[591907] Unexpected call to split method for User IO pages mapping vma
```

The last message indicates the VMA split being used to exploit the driver has been prevented.

Output by the application should look like the following for a driver that has been patched to fix CVE-2022-33917:

```
##### setup round 0x0
gpu_va = 0x41000
  gpu_va is cookie, doing real mmap...
CS_QUEUE_BIND: gpu_addr=0x7f2ad85f4000, handle=0
mmap_handle = 0x30000
victim_map = 0x7f2ad85f1000
##### setup round 0x1
gpu_va = 0x41000
  gpu_va is cookie, doing real mmap...

[...]

mali_test: SYSCHK(madvise(victim_maps[i], 0x1000, MADV_SEQUENTIAL)): Invalid
argument
```

What is the User Shared Memory used for?

User Shared Memory contains the majority of the data that the GPU uses for rendering and compute work, and is:

- mapped onto both the GPU and CPU
- is accessible by userspace

This includes:

- GPU Data including:
 - Textures that have been created from Application data
 - Shader programs that have been created from Application data
 - Job descriptors
 - Render targets/frame buffers
 - Cross-Queue Synchronization Objects ('CQS')
 - DMA-bufs
 - User process memory (such as `malloc()`d memory) imported into Kbase
- Command Buffers for communication with the CSF
 - These are mapped onto the CPU and GPU, but in a way that they are only available to the CSF part of the GPU.

Kbase is also able to access any memory available to the GPU. This is useful for:

- Kbase reading job descriptors and Command Buffers for outputting debugging information
- Updating job descriptors for:
 - Kbase-driven GPU memory allocations (Kbase "JIT" memory)
 - Kbase shrinking GPU intermediate storage ("Tiler Heap" memory) during the System's out-of-memory events
- Complex synchronization of work between Kbase and the GPU, using 'CQS' objects.

For more information on these, see the following sections:

- [Intra-process GPU/KCPU Queue/Userspace synchronization: CQS objects.](#)
- [General Memory management IOCTLs](#) for information on importing DMA-bufs and User process memory
- [Background on GPU Queues \("Commands Streams"\) and GPU Queue Groups \("Command Stream Groups"\)](#), for how Command Buffers are implemented.
- [Background on KCPU Queues](#), for:
 - The kinds of work that can be synchronized between the GPU and Kbase
 - Mapping/unmapping DMA-bufs and User process memory onto the GPU
 - Kbase-driven GPU memory allocation (Kbase "JIT" GPU memory).
- [GPU Intermediate storage \('JIT' and 'Tiler Heap' memory\) IOCTLs](#)

What is the Kbase Shared Memory used for?

Kbase Shared memory is data used by the GPU that is:

- Mapped onto the GPU
- Only mapped into kernelspace on the CPU
 - In some cases, this is only temporarily mapped onto the CPU.

This is used for:

- GPU MMU page tables
- CSF Firmware Image
- CSF Firmware Supporting data

GPU Registers are not classified in this way. Whilst they are memory-mapped, they do not live in the system's RAM.

Both Kbase Shared memory and (almost all) GPU registers are critical for operation of the GPU, and so cannot be mapped into userspace.

For more information on these, see the following sections:

- [Background on GPU Memory Management](#) for information about GPU MMU page tables
- [Using the Mali GPU Firmware](#) for information on the CSF Firmware Image

How do you access the Mali GPU Kernel Driver 'Kbase' through the U/K interface?

Kbase is accessed through syscalls (mainly IOCTLs) on a Mali file descriptor, usually obtained from opening `/dev/mali0`

Note: In some systems, there may be multiple mali instances, e.g. `/dev/mali0` and `/dev/mali1`

The IOCTLs are split into:

- A set of IOCTLs for "Job Manager" ('JM') GPUs
- A set of IOCTLs for Command Stream Frontend ('CSF') GPUs
- A set of IOCTLs common to both CSF and JM GPUs

Note: This document only covers the common set of IOCTLs and CSF IOCTLs.

Syscall Interface definitions for Kbase

For CSF-based GPUs, the IOCTLs/other user interfaces are described in:

- `$KDIR/include/uapi/gpu/arm/midgard/` :
 - `mali_kbase_ioctl.h` - general IOCTL definitions and parameter structures
 - `mali_kbase_mem_flags.h` - general flags and special values for memory IOCTLs
 - `mali_base_kernel.h` - general supplementary definitions and parameter structures
 - `mali_base_common_kernel.h` - context flag IOCTL flags, and other flags
 - `csf/`
 - `mali_kbase_csf_ioctl.h` - CSF-specific IOCTL definitions and parameter structures
 - `mali_base_csf_kernel.h` - CSF-specific supplementary definitions and parameter structures
 - `mali_kbase_csf_errors_dumpfault.h` - Definitions for "Dump on fault"
 - `mali_kbase_csf_mem_flags.h` - CSF-specific flags for memory IOCTLs
 - `gpu/`
 - `mali_kbase_gpu_coherency.h` - coherency definitions for gpu properties
 - `mali_kbase_gpu_id.h` - definitions for decoding GPU ID from gpu properties
 - `backend/csf/mali_kbase_gpu_regmap_csf` - user mapped register page definitions

Additional file descriptors can be created through the following syscalls:

- `KBASE_IOCTL_TLSTREAM_ACQUIRE` however this is only permitted from privileged processes
- `KBASE_IOCTL_KINSTR_PRFCNT_SETUP` - for performance monitoring

The relevant IOCTL definitions for KINSTR file descriptors are in:

- `$KDIR/include/uapi/gpu/arm/midgard/` :
 - `mali_kbase_hwcnt_reader.h` : `kinstr prfcnt` IOCTL definitions and parameter structures

- **Note:** `KBASE_IOCTL_HWCNT_READER_SETUP` and `KBASE_HWCNT_READER_*` are no longer available in driver versions r45p0 onwards.

Other files (such as those in the `backend/gpu` directory) are not relevant for accessing Kbase on CSF GPUs.

Kbase poll interface

This is used to indicate when a `read()` operation should be done on the Mali file descriptor, and unblocks when:

- There are regular events to be read from userspace
- There are error events to be read from userspace
- Kbase is requesting userspace to dump CPU queues, and userspace must respond by calling `KBASE_IOCTL_CS_CPU_QUEUE_DUMP`

Kbase read interface

This returns the event type and any supplementary data indicated by the `poll()` operation:

- data is of type `struct base_csf_notification`
- `type` member is:
 - `BASE_CSF_NOTIFICATION_EVENT` for regular events.
 - There is no data associated with this, instead userspace will normally re-evaluate anything blocked on a Cross-Queue Synchronization object (CQS).
 - `BASE_CSF_NOTIFICATION_GPU_QUEUE_GROUP_ERROR` for error events.
 - the `payload.csg_error` member contains more information. The `handle` member is the handle of the Command Stream Group (CSG) that had the error.
 - `BASE_CSF_NOTIFICATION_CPU_QUEUE_DUMP` to indicate that userspace needs to call `KBASE_IOCTL_CS_CPU_QUEUE_DUMP`.
 - There is no data associated with this

Kbase mmap interface

Kbase's mmap interface is implemented in two parts:

- `kbase_get_unmapped_area()` which is used to control where the address of memory is allocated
 - This is largely used for implementing 'SAME_VA' memory
- `kbase_context_mmap()` and can be supplied with one of the following in the `offset` parameter to `mmap()/mmap64()`:

`kbase_context_mmap()` handles a number of different cases for the `offset` parameter

- A 'special' handle, shifted up by `PAGE_SHIFT`.
- The PFN of an existing allocated GPU memory address.
 - This is used to map GPU memory to userspace, when its GPU address does not necessarily reside within the user process' address space.
 - It can also be used to make an additional mapping on an existing mapped GPU address.

The 'special' handles are as follows:

- `BASE_MEM_MMU_DUMP_HANDLE`
 - this is not available in production builds
- `BASEP_MEM_CSF_USER_REG_PAGE_HANDLE`
 - Maps a single read-only GPU HW page into userspace
 - Each process has the same view of this physical page
 - At present, it contains a `LATEST_FLUSH` register for userspace to program into Command Streams.
- `BASEP_MEM_CSF_USER_IO_PAGES_HANDLE..BASE_MEM_COOKIE_BASE - PAGE_SIZE`
 - A set of cookies for mapping in the user_io pages for GPU Command Streams (GPU Queues)
- `BASE_MEM_COOKIE_BASE..BASE_MEM_FIRST_FREE_ADDRESS - PAGE_SIZE`
 - A set of cookies for mapping in 'SAME_VA' memory

Variants of IOCTLs for compatibility

Kbase supports IOCTLs that use an older calling convention for backwards compatibility purposes. For example, IOCTL calls made against older releases of the IOCTL definitions where the IOCTL parameter structure was smaller.

These variants of IOCTLs have `_x_y` suffixes, where the following signify the version of the interface that was supported:

- `x` is the `BASE_UK_VERSION_MAJOR`
- `y` is the `BASE_UK_VERSION_MINOR`

For example, `KBASE_IOCTL_CS_TILER_HEAP_INIT_1_13` is the Tiler Heap initialization IOCTL up to U/K version 1.13, prior to the addition of the `buf_desc_va` IOCTL parameter structure member.

In general, these IOCTLs internally call the latest version of the IOCTL handler with the larger parameter structures, with extra parameters set to 0.

Not all versions are supported - older versions are deprecated periodically, with the support for the older IOCTL calling conventions removed from Kbase.

Note: The latest version of an IOCTL has no `_x_y` suffixes.

Initialization and Initial Querying IOCTLs

- `KBASE_IOCTL_VERSION_CHECK` is used to communicate the version of KBase's IOCTL interface that userspace is expecting.
 - KBase will return the version that it itself is using.
 - This IOCTL must be made before other general IOCTLs can be made.
- `KBASE_IOCTL_SET_FLAGS` is used to set a combination of `BASEP_CONTEXT` flags on the KBase context.
 - This IOCTL must be made before other general IOCTLs can be made, but after `KBASE_IOCTL_VERSION_CHECK`.
- `KBASE_IOCTL_KINSTR_PRFCNT_ENUM_INFO` enumerates performance counter information.
- `KBASE_IOCTL_GET_GPUPROPS` enumerates GPU properties.
 - The properties are serialized into the supplied buffer, using data type and data field information encoded in `KBASE_GPUPROP` macros.

`KBASE_IOCTL_KINSTR_PFCNT_ENUM_INFO` and `KBASE_IOCTL_GET_GPUPROPS` are called in two stages:

- The first call is to find out how large a buffer is required.
- The second call is to fill the buffer with information.

Note that `KBASE_IOCTL_VERSION_CHECK_RESERVED` is a legacy IOCTL that has no effect other than returning an error.

What are the Mali GPU Memory Management syscalls?

Background on GPU Memory Management

The GPU has its own Virtual Address space (up to 64-bits), separate from the CPU. The GPU has its own MMU that:

- Maps Virtual Address space to physical addresses
- Controls permissions (such as Read/Write/Execute)
- Controls cachability (Cached/non-cached)
- Controls sharability (not shared/shared within a GPU cluster/shared between GPU and CPU clusters)

The MMU page tables for the GPU are separate from those for the CPU, though they do follow a very similar format and structure to Arm's Aarch64 page tables.

Hence, memory to be used on the GPU must explicitly be mapped in through Kbase syscalls, which in turn will cause Kbase to update the GPU's MMU Page tables and issue corresponding GPU MMU and Cache operations (such as invalidation and cleaning).

GPU Memory allocated by Kbase can be mapped in, and memory allocated outside of kbase (such DMA-bufs or malloc()'d User process memory) can be mapped in by "Importing" it.

A process that opens the mali file descriptor is given a 'context' (Kbase context, or `kctx` in the code) that isolates its GPU memory from that of other contexts. There is usually only one Kbase context per process, but in case a process opens several different mali file descriptors, there would also be separate, isolated kbase contexts.

The GPU supports a limited number of Address Spaces. When scheduling work onto the GPU, it must also assign an address space number (known as Address Space ID - ASID) to the context. This ASID is the identifier for issuing MMU operations onto the relevant context. Each GPU Address space is separate and isolated from each other in the GPU HW, in the same way that kbase contexts are separate. For example, they have separate MMU Translation Table Base Addresses.

General Memory management IOCTLs

- `KBASE_IOCTL_MEM_ALLOC`: Allocates a new GPU memory region of 'native' type for general GPU use.
 - Supports partial physical backing and optional growth via GPU page-faults (`BASE_MEM_GROW_ON_GPF`).
 - Accepts various `BASE_MEM` flags to control GPU/CPU access and mapping behaviors.
- `KBASE_IOCTL_MEM_ALLOC_EX`: Allocates GPU memory similar to `KBASE_IOCTL_MEM_ALLOC` but supports fixed GPU virtual addresses.
 - `BASE_MEM_FIXED` and `fixed_address` can be used to place memory precisely in FIXED zones.
- `KBASE_IOCTL_MEM_IMPORT`: Imports external memory into the GPU, either from dma-buf (UMM) or userspace buffers (USER_BUF).
 - USER_BUF imports allow memory such as `malloc()` allocations to become accessible to the GPU.
 - Imported memory has restrictions on which `BASE_MEM` flags may be applied.

Copyright © 2025 Arm Limited (or its affiliates). All rights reserved.

Public

- **KBASE_IOCTL_MEM_ALIAS**: Creates an aliased GPU memory region composed of existing source allocations.
 - Allows for complex layouts including gaps/strides and subranges from source allocations.
 - Special mapping behavior can be specified with **BASEP_MEM_WRITE_ALLOC_PAGES_HANDLE**.
- **KBASE_IOCTL_MEM_FREE**: Frees a previously allocated or imported GPU memory region.
 - For 'SAME_VA' memory, freeing is typically done with **munmap()**.
 - Otherwise, freeing uses **KBASE_IOCTL_MEM_FREE** directly with a GPU address or memory cookie.
- **KBASE_IOCTL_MEM_SYNC**: Issues a CPU cache coherency operation on a GPU memory region.
 - Used for CPU-cached memory to ensure data consistency between CPU and GPU.
 - Some memory types, such as **USER_BUFFER** imports, are enforced to be CPU-cached and thus may require syncs if access is made through the GPU mapping, instead of the original **USER_BUFFER** pointer
- **KBASE_IOCTL_MEM_COMMIT**: Adjusts the amount of physical memory backing a GPU memory region.
 - Only allowed on 'native' memory regions that are not shrinkable, aliased, imported, or kernel-mapped.
 - Enforced checks such as **BASEP_MEM_NO_USER_FREE** help prevent invalid commit operations on restricted memory types.
- **KBASE_IOCTL_MEM_FLAGS_CHANGE**: Modifies certain memory flags after allocation, such as **BASE_MEM_DONT_NEED** and coherency settings.
 - Allows marking memory as shrinkable/unshrinkable (similar restrictions to **KBASE_IOCTL_MEM_COMMIT** apply).
 - For imported memory, allows changing the coherency of the memory within the GPU, and (on Ace-Lite systems) between GPU and CPU.
- **KBASE_IOCTL_READ_USER_PAGE**: Reads values directly from the **BASEP_MEM_CSF_USER_REG_PAGE_HANDLE** user-mappable GPU register page.
 - Currently only supports reading the **LATEST_FLUSH** register via the **offset** parameter.
 - The value returned is useful for cache-flushing operations in CSF firmware instructions.
- **KBASE_IOCTL_MEM_QUERY** to query one property of a region:
 - its physical backing amount
 - its virtual size
 - its flags (not all flags may be returned by the query)
- **KBASE_IOCTL_MEM_EXEC_INIT**
 - This is no longer used on recent KBase driver releases, and is essentially a no-op.
- **KBASE_IOCTL_MEM_FIND_CPU_OFFSET** to determine where the offset of a CPU Virtual address (from the start of the region) is within a CPU mapping of a region.
- **KBASE_IOCTL_MEM_FIND_GPU_START_AND_OFFSET** to determine the offset and GPU start Virtual address of a GPU region, for a particular GPU Virtual address.
- **KBASE_IOCTL_MEM_PROFILE_ADD** for adding a buffer of memory to Kbase to make it available via debugfs.
 - This is usually used for profiling memory usage by the userspace driver, but Kbase doesn't parse or modify the contents of the buffer.

Imported Memory Mapping IOCTLs

- **KBASE_IOCTL_STICKY_RESOURCE_MAP**: Immediately maps an imported "User buf" memory onto the GPU for longer-term access.
 - Increases a user mapping reference count; memory is pinned first if it hasn't already been.
 - The first reference count will also ensure cache coherency operations are done.
- **KBASE_IOCTL_STICKY_RESOURCE_UNMAP**: Immediately unmaps a previously mapped imported "User buf" memory from the GPU.
 - Decreases the user mapping reference count, but doesn't unpin memory (memory is pinned until the import is freed with **KBASE_IOCTL_MEM_FREE**)
 - The last reference count released ensures that cache coherency and GPU MMU teardown occur properly.

GPU Intermediate storage ('JIT' and 'Tiler Heap' memory) IOCTLs

- **KBASE_IOCTL_MEM_JIT_INIT**: Initializes the JIT memory zone for 64-bit userspace processes before any allocations occur.
 - Partitions a JIT-specific region from the 'SAME_VA' zone for later JIT memory allocations.
 - Necessary for enabling efficient GPU-managed memory used in operations such as Tile-based rendering.
- **KBASE_IOCTL_CS_TILER_HEAP_INIT**: Initializes a Tiler Heap for the GPU with specified chunk size and number of chunks.
 - Returns a heap context GPU address and the first chunk header address; requires prior JIT zone setup with **KBASE_IOCTL_MEM_JIT_INIT**.
 - Optionally supports supplying a pointer to a GPU Buffer Descriptor, for shrinker-based memory freeing during low-memory events.
- **KBASE_IOCTL_CS_TILER_HEAP_SIZE**: Queries size and usage statistics for an existing Tiler Heap from its heap context.
 - Helps monitor memory usage and chunk allocation in the Tiler Heap.
- **KBASE_IOCTL_CS_TILER_HEAP_TERM**: Terminates a Tiler Heap, freeing associated memory based on the heap context address.
 - Ensures proper cleanup of Tiler Heap allocations from the JIT zone.

What are the GPU Queue Management Syscalls?

Background on GPU Queues ("Commands Streams") and GPU Queue Groups ("Command Stream Groups")

GPU Queues are the shared memory interface for providing commands via Command Buffers to the GPU CSF Firmware.

GPU Queues may be grouped into Command Stream Groups (CSGs, or just 'Group's), by 'binding' them to the group. The group of queues are scheduled together on the GPU, and may share extra information and synchronization between them directly. Queues programmed onto the GPU via a GPU group are assigned a Command Stream Interface (CSI).

Each group can be assigned a limited number of queues - at least `BASEP_GPU_QUEUE_PER_QUEUE_GROUP_MIN` (8) but potentially a GPU in future could expose up to `BASEP_GPU_QUEUE_PER_QUEUE_GROUP_MAX` (32) queues.

The interface to the Queues is accessible after binding the Queue to a Group, exposed on 3 pages per Queue:

- A GPU Hardware Register Mapped Doorbell Page
 - A "Doorbell" is essentially a GPU CSF Firmware Interrupt that the CPU can raise on the GPU.
 - This contains a single 32-bit entry for requesting the Queue to be started/resumed, by writing the value 0x1 to it.
- A GPU Virtual Memory Input page (information to send to the CSF Firmware), containing:
 - `CS_INSERT *` virtual registers
 - `CS_EXTRACT_INIT *` virtual registers
- A GPU Virtual Memory Output page (information received from the CSF Firmware), containing:
 - `CS_EXTRACT *` virtual registers
 - `CS_ACTIVE` virtual register, which is used to determine if the queue can be started via the "Doorbell", or (in the inactive case) whether the `KBASE_IOCTL_CS_QUEUE_KICK` ioctl must be used.

`CS_INSERT *` and `CS_EXTRACT *` indicate offsets relative to a Queue's GPU Memory Buffer, modulo the size of the Queue's GPU Memory Buffer. Hence, the Queue's GPU Memory Buffer is treated as a ringbuffer.

The Input and Output pages are also mapped into the kernel, so that Kbase may inspect them for GPU Scheduling reasons, or for implementing suspend and resume of GPU work.

When the GPU is powered off, access to the GPU HW "Doorbell" register page is not possible, but userspace may be unaware of that. In this case, Kbase will remap all Queues' Doorbell pages across all contexts in the system to a `dummy_db_page`. The writes from the page are discarded, as they are not required whilst the GPU is powered off. Instead, Kbase will signal the CSF Firmware to restart the queues once the GPU has powered up again, instead of requiring userspace to do so.

Since there are a limited number of GPU HW "Doorbell" register pages, Kbase will also do this remapping of Queues' Doorbell pages to the `dummy_db_page` whenever the Queue is

stopped on the GPU HW. This frees up the HW Doorbell register pages for other Queues to be scheduled on the GPU. Again, writes to the Doorbell page are not required whilst the GPU Queue is stopped.

CSF Firmware Instructions in the Queue's Buffer

The contents of the GPU Queue's Buffer is a set of CSF Firmware instructions.

The following code may be useful in understanding the use of these registers, and interpreting the CSF Firmware instructions:

- `kbasep_csf_dump_active_queue_sync_info()`, which dumps the queue CSF Firmware instructions nearby (prior to and following) the `CS_EXTRACT` offset.
 - `kbasep_csf_print_gpu_sync_op()` decodes the CSF Firmware Synchronization instructions.
- OpCode list and formats of each instruction can be found under `CS_Opcode` enum and corresponding struct definitions in <https://gitlab.freedesktop.org/mesa/mesa/-/blob/main/src/panfrost/lib/genxml/v10.xml>

GPU Queue Management IOCTLs

- `KBASE_IOCTL_CS_QUEUE_REGISTER`: Registers a GPU queue with a given ring buffer ('queue buffer') memory region and size.
 - The queue buffer must be a valid GPU virtual address with a power-of-2 size between `CS_RING_BUFFER_MIN_SIZE` and `CS_RING_BUFFER_MAX_SIZE`.
 - Initializes the queue in an unbound state and marks the buffer as "No User Free" to prevent premature release.
- `KBASE_IOCTL_CS_QUEUE_REGISTER_EX`: Extended version of `CS_QUEUE_REGISTER` supporting an additional GPU address for trace buffers.
 - Allows enabling GPU tracing alongside standard queue setup via optional parameters.
- `KBASE_IOCTL_CS_QUEUE_TERMINATE`: Terminates an existing GPU queue and cleans up associated GPU and kernel resources.
 - Stops GPU execution, unbinds from any group, cancels workers, and unregisters the queue buffer.
 - Userspace retains ownership of the queue buffer and must manage its reuse or release.
- `KBASE_IOCTL_CS_QUEUE_GROUP_CREATE`: Creates a GPU Queue Group (a Command Stream Group - 'CSG') with memory setup based on input parameters.
 - Returns a `group_handle` to identify the group; optionally supports Deferred Vertex Shading ('DVS') buffers for supported GPUs.
 - Allocates suspend buffers as needed for group execution, and (for userspace clients identified as r53p0 or newer) GPU Queue User IO pages.
- `KBASE_IOCTL_CS_QUEUE_GROUP_TERMINATE`: Terminates a GPU Queue Group and releases associated resources.
 - Deschedules the group, unbinds its queues, frees GPU memory, and removes the group from tracking tables.
 - Userspace must still manage separately owned memory such as the DVS buffer after termination.
- `KBASE_IOCTL_CS_QUEUE_BIND`: Begins a 2-stage binding of a GPU queue to a queue group to enable scheduling on the GPU.
 - Returns a temporary cookie; the 2nd stage of binding is completed by `mmap()` to map "User IO" pages for the queue (similar to 'SAME_VA' memory mapping)

- Manages setup for GPU Command Stream Interfaces ('CSI's) and validates `csi_index` to prevent collisions within the group.
- **`KBASE_IOCTL_CS_QUEUE_KICK`**: Requests scheduling of a GPU queue and its bound group onto the GPU hardware.
 - Used when a queue is descheduled or has never been scheduled, to ensure it gets picked up by the CSF firmware.
 - Scheduling happens asynchronously and may be delayed by GPU workload and priority.
- **`KBASE_IOCTL_QUEUE_GROUP_CLEAR_FAULTS`**: Clears reported faults on one or more GPU queues to re-enable error reporting.
 - Takes an array of registered queue buffer addresses to clear faults for each queue.
 - Necessary since each queue can only report a single error at a time via `read()`.
- **`KBASE_IOCTL_CS_GET_GLB_IFACE`**: Retrieves global information about GPU groups and streams, including resource limits and feature support.
 - Requires a two-step call: first to query sizes, then to fetch group and stream details into allocated memory.
 - Provides necessary data for resource management and debugging, such as stream counts and feature flags.

What are the Synchronization Operations and Syscalls?

Background on Synchronization in Kbase

There are two ways of synchronizing work in Kbase:

- Cross Queue Synchronization ('CQS') Objects, which allow synchronizing work within a context, between:
 - GPU Queues
 - KCPU Queues
 - The userspace client of the context
- Linux's Fences, which allow synchronizing work between different contexts/processes
 - Kbase supports Linux's Fence synchronization through KCPU Queues.

For more information on KCPU Queues, refer to [What are KCPU Queue Management Syscalls?](#).

Intra-process GPU/KCPU Queue/Userspace synchronization: CQS objects

GPU Queues, KCPU Queues, and Userspace can be synchronized with Cross Queue Synchronization ('CQS') objects.

CQS objects are a pair of values in memory:

- Either a 32-bit ('Sync32' objects) or 64-bit ('Sync64' objects) unsigned value
- A 32-bit error value

GPU Queues and KBase's KCPU Queues use the exact same format for the CQS. In addition, the format is available to userspace. And so:

- A GPU Queue can signal or wait on another GPU Queue, a KCPU Queue, or the Userspace process
- A KCPU Queue can signal or wait on another KCPU Queue, or a GPU Queue, or the Userspace process
- The Userspace process can signal or wait on a KCPU Queue, or a GPU Queue

Note: In the case of userspace, it can only signal or wait on CQS in its own context. There is no cross-process synchronization using CQS.

The CQS Object's "error value" can be used to propagate previous errors between GPU Queues, KCPU Queues, and userspace.

Memory for CQS objects is allocated using the `BASE_MEM_CSF_EVENT` flag. Such memory is permanently mapped into the kernel during its lifetime using `vmap()`. This allows Kbase to quickly obtain access to the memory to evaluate or update it.

The CQS Objects' memory must be aligned when suballocating from the `BASE_MEM_CSF_EVENT` memory:

- 32-bit CQS must be aligned to 8 bytes.
- 64-bit CQS must be aligned to 16 bytes.

Memory for CQS objects is either mapped as:

- CPU and GPU uncached
- or in ACE-lite Systems, as CPU and GPU cached and Outer-sharable

Inter-process synchronization: Fences

Fences are available on the system with recent versions of Android enabling `CONFIG_SYNC_FILE` by default, via `CONFIG_DRM`.

Fences are referred to by their file descriptor, which may be passed to other processes using standard Linux APIs.

Kbase can wait upon and signal Fences using KCPU Queue commands.

The GPU can be signalled on completion of a Fence by combining a KCPU `FENCE_WAIT` command with a KCPU `CQS_SET/CQS_SET_OPERATION` command, with a corresponding `SYNC_WAIT` CSF Firmware instruction enqueued on the GPU queue.

Similarly, a Fence can be signalled by a GPU queue by first using a `SYNC_SET` CSF Firmware instruction, and then combining a KCPU `CQS_WAIT/CQS_WAIT_OPERATION` command with a KCPU `FENCE_SIGNAL` command.

For more information on KCPU Commands, refer to [Background on KCPU Queues](#).

Synchronization Related Ioctls

- `KBASE_IOCTL_CS_EVENT_SIGNAL`: Signals a CQS object to notify Kbase or the GPU.
 - Used by userspace to trigger event notifications to KCPU Queues or the GPU itself.
- `KBASE_IOCTL_STREAM_CREATE`: Creates a synchronization stream for Fence (`struct base_fence`) objects.
 - Note that the `stream_fd` member of `struct base_fence` is ignored in KCPU Fence commands, so this IOCTL is not required.
- `KBASE_IOCTL_FENCE_VALIDATE`: Validates whether a given file descriptor refers to a valid `struct dma_fence` object.
 - Returns 0 if valid, or a negative error code otherwise.

What are KCPU Queue Management Syscalls?

Background on KCPU Queues

Similar to [GPU Queues](#), Kernel-space-CPU Command Queues ("KCPU Queues") allow userspace to submit commands for processing. Instead of executing on the GPU, these commands are executed on the CPU in kernel space by Kbase. The queues are executed strictly in order, and independent of other KCPU queues (unless explicitly synchronized).

KCPU Queues support a number of different commands, defined by the `enum base_kcpu_command_type`.

Each context supports up to 256 KCPU Queues, and each queue may be enqueued with up to 256 active commands.

KCPU Queues may be synchronized with:

- [CQS Objects](#) for synchronization within a context, such as with GPU queues or other KCPU queues
- [Fences](#), for synchronization between contexts/processes

A KCPU Queue can be blocked on any of:

- CQS Objects
- Fences
- JIT Memory Allocations

When blocked, it makes no progress on processing further commands. More commands may still be enqueued (up to the limit of the queue).

Some KCPU Commands take some immediate action on enqueueing, even when the KCPU Queue the command is enqueued on is blocked. Refer to the different [KCPU Queues Commands](#) for more information.

KCPU Queues may have an error condition that can optionally be propagated:

- from CQS objects or Fences
- to other CQS objects

KCPU Queues IOCTLs

- `KBASE_IOCTL_KCPU_QUEUE_CREATE`: Creates a new KCPU Queue for submitting CPU-side GPU management commands.
 - Returns an identifier (`id`) used to reference the queue in later operations.
- `KBASE_IOCTL_KCPU_QUEUE_DELETE`: Deletes a KCPU Queue and drains any remaining blocked commands.
 - Ensures cleanup of resources and unblocking of dependent synchronization objects.
- `KBASE_IOCTL_KCPU_QUEUE_ENQUEUE`: Enqueues a single command onto a KCPU Queue for later asynchronous execution.
 - Commands are specified via a `struct base_kcpu_command` and must currently be submitted one at a time.

Information on specific KCPU commands can be found below

KCPU Queues Commands

- **BASE_KCPU_COMMAND_TYPE_FENCE_WAIT**: Waits on a Fence and blocks the KCPU queue until the Fence is signaled.
 - Further commands can still be enqueued while waiting; the queue will not progress until the Fence signals.
- **BASE_KCPU_COMMAND_TYPE_FENCE_SIGNAL**: Creates a Fence that will be signaled once prior blocking operations complete.
 - A timeout mechanism ensures progress even if dependent commands block indefinitely.
- **BASE_KCPU_COMMAND_TYPE_CQS_WAIT**: Waits for specified 32-bit CQS objects to exceed given values
 - Up to 32 CQS wait conditions can be handled per command; queue progress halts until all waits are satisfied.
 - Optionally can propagate error conditions from individual CQS objects via the **inherit_err_flags** bitmask.
- **BASE_KCPU_COMMAND_TYPE_CQS_SET**: Increments specified 32-bit CQS objects
 - Updates notify the GPU CSF Firmware so dependent GPU queues can also be unblocked.
 - Always propagates queue error conditions to the CQS objects.
- **BASE_KCPU_COMMAND_TYPE_CQS_WAIT_OPERATION**: Waits on 32/64-bit CQS objects with more flexible comparison operations.
 - Allows waits such as greater-than, equal, not-equal, etc., specified per CQS object.
 - As with **BASE_KCPU_COMMAND_TYPE_CQS_WAIT**, can optionally propagate error conditions via the **inherit_err_flags** bitmask.
- **BASE_KCPU_COMMAND_TYPE_CQS_SET_OPERATION**: Updates 32/64-bit CQS objects using flexible arithmetic or logical set operations.
 - Supports operations such as add, subtract, bitwise operations.
 - Always propagates queue error conditions to the CQS objects.
- **BASE_KCPU_COMMAND_TYPE_ERROR_BARRIER**: Clears the current error condition of the KCPU queue.
 - Executed only after processing prior blocking operations.
 - Any previous errors on the queue could be propagated to CQS objects up until the error barrier command.
- **BASE_KCPU_COMMAND_TYPE_MAP_IMPORT**: Maps an imported "User Buf" memory onto the GPU and increments its user mapping refcount.
 - Involves Linux's DMA API mapping, cache maintenance, and updating GPU MMU entries at first mapping.
 - On enqueueing the command the imported User Buffer will be 'pinned' immediately (if not already pinned), regardless of whether there are any previous blocking commands.
- **BASE_KCPU_COMMAND_TYPE_UNMAP_IMPORT**: Decrements the user mapping refcount for imported "User Buf" memory and unmaps from the GPU if counts reach zero.
 - Cache operations and GPU MMU unmapping happen when the final refcount is released.
 - The imported user buffer is *not* unpinned on unmapping/refcount(s) becoming zero, and instead remains pinned until the imported region is removed.
- **BASE_KCPU_COMMAND_TYPE_UNMAP_IMPORT_FORCE**: Forces an imported memory's user mapping refcount to zero, triggering unmapping if applicable.
 - Works for "User Buf" memory; DMA-buf imports maintain a separate refcount and are not unmapped this way.

- **BASE_KCPU_COMMAND_TYPE_JIT_ALLOC**: Allocates one or more "JIT" GPU memory regions, writing addresses into GPU data structures (not through the original command parameter structure).
 - Allocation is delayed if necessary to enforce in-flight JIT memory limits, ensuring atomic handling of all allocations.
 - This command may block further commands on the same KCPU Queue from being processed, until the allocation can be made.
- **BASE_KCPU_COMMAND_TYPE_JIT_FREE**: Frees one or more "JIT" GPU memory regions back to the JIT pool.
 - Enables memory reuse or freeing during out-of-memory conditions; supports trimming based on configured levels.
 - The JIT memory regions to free are identified by their **id** (given to **BASE_KCPU_COMMAND_TYPE_JIT_ALLOC**) instead of GPU address. These **ids** are allocated by userspace.

Details of Miscellaneous Mali syscalls

- **KBASE_IOCTL_DISJOINT_QUERY** returns a single value **counter**.
 - This counter gets incremented at certain "disjoint operations" in the driver.
 - Refer to the [EXT_disjoint_timer_query](#) OpenGL ES extension for more information on "disjoint operations"
- **KBASE_IOCTL_GET_DDK_VERSION** returns Kbase's version.
 - When **version_buffer** is NULL, it will return the size (including NUL character) of the string as the return value of **ioctl()**.
 - When **size** is given, it will copy the version string into the user-provided **version_buffer**
- **KBASE_IOCTL_GET_CONTEXT_ID** returns a the context's **id** member.
 - This is a unique sequential number assigned to each kctx to identify it.
 - Uniqueness is only guaranteed for the first 2^{32} contexts to have ever been created.
- **KBASE_IOCTL_GET_CPU_GPU_TIMEINFO**: Captures CPU and/or GPU timestamps, for example for time correlation purposes.
 - Supports querying the GPU cycle counter, GPU timestamp counter, and CPU monotonic clock.
 - **BASE_TIMEINFO...** flags control what combinations of counters are retrieved.
- **KBASE_IOCTL_CS_CPU_QUEUE_DUMP**: Pass a userspace-generated string describing CPU-side GPU queues to Kbase.
 - Called after userspace receives **BASE_CSF_NOTIFICATION_CPU_QUEUE_DUMP** (during a KCPU fence-signal timeout), or when triggered by debugfs access.
 - The provided string is captured verbatim for debugging purposes, and is not parsed or inspected by KBase. it is discarded if sent at unintended times.
- **KBASE_IOCTL_CONTEXT_PRIORITY_CHECK** supplies a priority to the **priority_control_manager_device** module ('PCM module'), and returns an adjusted priority that the PCM module is willing to grant the calling process.
 - Refer to **pcm_scheduler_priority_check** in:

```
$KDIR/include/linux/priority_control_manager.h
```

- **KBASE_IOCTL_SET_LIMITED_CORE_COUNT** is a legacy IOCTL that is only useful on Job Manager GPUs. It has no effect on CSF GPUs.
- **KBASE_IOCTL_TLSTREAM_ACQUIRE** is used to acquire a single Kernel Timeline Stream for instrumentation purposes. Only one process may acquire it at a time across the system. However, privileges are required to do this.
- **KBASE_IOCTL_TLSTREAM_FLUSH** will flush the Kernel Timeline Stream, and is intended to have no effect when the no process has currently acquired it.

Using the Mali GPU Firmware

How do you access the Command Stream Frontend (CSF) and GPU Firmware, through Command Buffers?

Command Buffers are created using GPU Queues, and are placed in system memory.

The memory for them is only accessible once the GPU Queue has been bound to a GPU Queue Group ('CSG's).

The contents of the Command Buffers are CSF Firmware Instructions.

The GPU Queues are then executed after the CSGs have been scheduled onto the GPU.

Command Buffers communicate back to the CPU using:

- `*_REQ` / `*_ACK` GPU registers, available only to Kbase/kernel-space
- Cross-Queue Synchronization Objects, available both to Kbase and to Userspace.

For more information on these, refer to:

- [Background on GPU Queues \("Commands Streams"\) and GPU Queue Groups \("Command Stream Groups"\)](#), for:
 - how to create and access Command Buffers (GPU Queues)
 - how to start queues either with the "Doorbell" page or `KBASE_IOCTL_CS_QUEUE_KICK`, as appropriate.
- [CSF Firmware Instructions in the Queue's Buffer](#), for writing the contents of Command Buffers.
- [Intra-process GPU/KCPU Queue/Userspace synchronization: CQS objects](#).

Decoding the Firmware Image

The `mali_csffw.bin` Firmware Image is split up into a number of sections.

The Firmware Image parsing begins in `kbase_csf_firmware_load_init()`, after the `Loading Mali firmware` notice in `mali_kbase_csf_firmware.c`.

Each section begins with a `u32` header followed by extra information depending on the type of the section.

The different types of section are parsed using the `load_firmware_entry()` function which uses a switch statement to select the relevant parsing of each of the `CSF_FIRMWARE_ENTRY_TYPE_...` section types.

Patching Kbase to decode the Firmware Image Sections in a Virtual Environment

The Kbase source can be used to decode the Firmware Image Sections.

Note: in "No Mali" builds used in a Virtual Environment, the CSF Firmware is not loaded.

For investigation purposes Kbase may be modified to compile in the Firmware parsing functionality enabled in a Virtual Environment.

Note: using this method, startup of the driver will fail. Hence, this is only useful for investigating the Firmware Image itself.

Modify `drivers/gpu/arm/midgard/csf/Kbuild` to compile in the firmware parsing code from `mali_kbase_csf_firmware.c` in `CONFIG_MALI_NO_MALI=y` builds:

```
ifeq ($(CONFIG_MALI_NO_MALI),y)
#### REMOVE "_no_mali" ####
mali_kbase-y += csf/mali_kbase_csf_firmware.o
mali_kbase-y += csf/mali_kbase_csf_fw_io_no_mali.o
else
mali_kbase-y += csf/mali_kbase_csf_firmware.o
```

Add in some code in `parse_memory_setup_entry()` to output the relevant information:

```
printk("FW interface mem '%s' offset 0x%.5x data_start 0x%.5x data_end "
"0x%.5x loads at CSF addr 0x%.8x..0x%.8x flags=0x%.8x EXEC=%s\n",
name, (const u8*)entry - fw->data, data_start, data_end,
virtual_start, virtual_end, flags,
(flags & CSF_FIRMWARE_ENTRY_EXECUTE) ? "yes" : "no");
```

To allow the driver to startup on `CONFIG_OF=n` builds (in x86/"Simulated Platform Device" configurations), add the following to `kbase_gpu_device_create()`:

```
kbdev->reg_size = 0x200000ul;
```

Recompile `mali_kbase.ko`.

The CSF Firmware should then be installed into `/lib/firmware` in the Virtual Environment:

```
sudo cp -a mali_csffw.bin /lib/firmware/mali_csffw.bin
```

Kbase will load the firmware on running the first application using the GPU. The libGPUCounters example can be used for this:

```
sudo insmod $KDIR/drivers/gpu/arm/midgard/mali_kbase.ko

# Run a Mali Application to trigger firmware load
../libGPUCounters/build/examples/api-example
```

Example output of the debugging:

```
mali mali.0: Probed as mali0
mali mali.0: Loading Mali firmware 0x30a0000
FW interface mem '' offset 0x00018 data_start 0x0114c data_end 0x011ec loads
at CSF addr 0x00400000..0x00410000 flags=0x00000009 EXEC=no
FW interface mem '' offset 0x00038 data_start 0x011f0 data_end 0x0282c loads
at CSF addr 0x00410000..0x00420000 flags=0x00000009 EXEC=no
FW interface mem '' offset 0x00058 data_start 0x0114c data_end 0x011ec loads
at CSF addr 0x00000000..0x00010000 flags=0x00000009 EXEC=no
FW interface mem '' offset 0x00078 data_start 0x0282c data_end 0x14d2c loads
at CSF addr 0x00800000..0x00820000 flags=0x0000000d EXEC=yes
```

Note: warnings will follow this, but these can be ignored for the purpose of decoding the CSF Firmware. However the driver will not be functional in this build.

Decoding the Firmware's Arm Instructions

The CSF itself executes ARMv7-M Thumb instructions. In the firmware image, these can be found in the following section:

- Section of type `CSF_FIRMWARE_ENTRY_TYPE_INTERFACE` (parsed by `parse_memory_setup_entry()`)
- Has `CSF_FIRMWARE_ENTRY_EXECUTE` flag set in `flags`

In the example output above, this is at firmware image offset **0x0282c** to **0x14d2c**.

The ARMv7-M Vector table is also required to decode the entry point into the firmware. This can be found in the following section:

- Section of type `CSF_FIRMWARE_ENTRY_TYPE_INTERFACE`
- Non-executable memory (does *not* have `CSF_FIRMWARE_ENTRY_EXECUTE` flag set in `flags`)
- Is located at CSF address **0x0**

Offset **0x4** within this section gives the address of the reset handler, modified with bit 0 set to 1 as per the ARMv7-M specification.

In the example output above, the Vector table is at firmware image offset **0x0114c** to **0x011ec**, with the reset handler located at offset **0x01150**

Decoding the example Firmware binary can then be done as follows:

```
hexdump -e '"%08_ax: " 1/4 "%08x " "\n"' mali_csffw.bin
...
0000114c: 02036000 ; SP_main - reset of the Main stack pointer
00001150: 00801fcd ; Reset handler CSF address, +bit0 set for EPSR.T
00001154: 00802095 ; NMI handler CSF address, +bit0 set for EPSR.T
```

Using the above Executable Memory firmware image offset and the Reset handler CSF address, the Firmware's entrypoint can be found at the following firmware image offset:

```
0x282c + ((0x00801fcd - 1) - 0x00800000) == 0x47f8
```

This may be disassembled with:

```
arm-linux-gnueabi-hf-objdump -D --start-address=$((0x47f8)) --stop-address=$((0x14d2c)) -m armv7e-m -Mforce-thumb -b binary mali_csffw.bin
```

This gives output starting with:

```
mali_csffw.bin:      file format binary
```

```
Disassembly of section .data:
```

```
000047f8 <.data+0x47f8>:
```

```
    47f8:    b672        cpsid    i
```

Note: in this example Firmware, the range **0x0282c..0x14d2c** contains all of the code, with some relevant code present at a lower address than the reset handler.

Proprietary notice

This document is protected by copyright and other related rights and the practice or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of the document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any use, duplication or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership

relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. All rights reserved. Please follow Arm's trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.
110 Fulbourn Road, Cambridge, England CB1 9NJ.
(PRE-1121-V1.0)